
Programmering eksamensprojekt

Real time messaging app

Projekt navn: Real time messaging app

Udarbejdet af: Melanie Laursen

Klasse: Y22

Gruppenavn: Gruppe 7

Vejleder: Alina Ostapenko-Bozhnova

Uddannelsessted: HTX Thisted - EUC Nordvest

Abstract

Dette projekt er et chat app som er udviklet i Python, hvor man kan tilføje andre der bruger programmet som venner og skrive med dem. Programmet er simpelt med et simpelt design.

Indhold

Problemformulering	4
Funktionsbeskrivelse	4
Dokumentation af selve programmet	4
Test af programmet	6
Konklusion	6
Bilag	8

Problemformulering

Hvordan kan man lave en real time messaging app hvor man kan skrive beskeder til andre personer og laver grupper hvor flere personer kan skrive sammen. Og hvordan kan man lave det sådan at man kan oprette profiler til at skrive sammen sådan så det ikke er tilfældige mennesker man skriver med?

Funktionsbeskrivelse

Når man tænker på en besked app, er der nogle grundlæggende funktioner som er nødvendige at have, for at appen kan bruges. Først og fremmest skal brugeren kunne oprette en profil man kan logge ind på, sådan at man som bruger har en form for personlig identitet på appen og i systemet. Dette indebærer typisk et brugernavn og en adgangskode, dette kan også udvides til at inkludere ens nuværende aktive status eller profilbilleder for at kunne give et mere personligt præg.

En anden vigtig funktion for sådan en app er at kunne sende private beskeder til andre personer, disse private beskeder kalder man for DM's af de engelske ord 'Direct Messages', det handler om at kunne sende en besked fra bruger til en anden bruger. Dette er en kernefunktion i enhver besked app, da det tillader direkte og fortrolig kommunikation mellem to personer. Udover dette bør brugeren også have mulighed for at kunne oprette og blive en del af grupper hvor der er flere brugere der skriver inde i samme chat, dette gør besked apps en del mere fleksible i sociale sammenhænge men også i arbejdsmæssige sammenhænge. Disse funktioner er det jeg har haft størst fokus på, oprette en profil, skrive med andre personer, og kunne oprette en gruppe for at skrive flere personer ad gangen i en chat på samme tid.

Nogle af de funktioner man kunne have med, er funktioner som at have profilbilleder, notifikationer omkring beskeder, aktivitet på appen og redigering og sletning af beskeder, kunne sende billeder og filer. Disse funktioner er funktioner der kunne overvejes at blive implementeret hvis tiden var til det, da det generelt set ville give et bedre look til appen da der er flere funktioner at kunne bruge.

En lille del af de funktioner beskrevet er en del af appen, dog så er det lidt mere krævende funktioner at implementere resten af dem, og med tiden givet har det kun været overvejelser der kunne tænkes over. Hvis der var mere tid at kunne bruge på at lave det, så ville der være blevet implementeret flere funktioner for selve appen.

Brugeren kan i dette program søge på personers navne og tilføje dem som ven for at kunne skrive med dem. Programmet i sig selv er justerbar og er over i den mørke side af dens farver, med mørkeblå farver og hvis.

Dokumentation af selve programmet

Grundet at det er en chat app som jeg har lavet, så er der rigtig mange definitioner i min programmering, og ikke lige så mange variabler. Dog så kan mange af disse variabler godt bruge noget forklaring for at forstå det helt præcist. Jeg tager her 4 eksempler, dog lidt lange eksempler,

for at bedre at kunne forklare det jeg har lavet. Appen er lavet i Python med brug af tkinter for at vise tekstfelter, knapper og lister.

Fra linje 38 til linje 60 i koden forklare det helt basalt, at når man starter koden op så får man at vide at man skal skrive sit brugernavn i et felt, programmet vil derefter tjekke om der er skrevet noget eller ej. Hvis feltet er tomt, får brugeren en fejlbesked, hvis brugernavnet ikke findes endnu, vil der blive oprettet en ny venne- og beskedliste. Hvis navnet allerede findes, vil man blive logget ind og få adgang til de chats man allerede har skrevet. Uanset om man er en ny bruger eller tilbagevendende bruger, vil navnet blive gemt så programmet ved hvem der er logget ind. Efter brugernavnet er godkendt kommer man videre til hovedskærmen.

Hovedskærmen består af 2 dele, til venstre er der en liste med alle brugerens venner og oplysninger om hvem der logget ind lige ovenover. Til højre for listen vises selve chatten, når brugeren trykker på en vens navn, vises de tidligere beskeder mellem dem og man kan i bunden skrive flere beskeder.

Fra linje 74 til linje 94 er der skrevet for hvordan man tilføjer en ven. Selve koden der viser bare at man skal trykke på en knap for at tilføje en ven, hvor man derefter skal skrive deres navn ind for at de kan blive venner. Selve programmet tjekker et par ting når man gør dette:

Den tjekker om et navn er skrevet ind, at man ikke prøver at skrive sig selv ind, den tjekker om brugeren findes, og om man allerede er venner eller ej. Hvis alt er i orden og der ikke er sket nogen fejl, så bliver begge brugere tilføjet til hinandens venne-liste, og der bliver derefter oprettet en tom chat hvor de kan skrive sammen i. Listen over venner bliver efter dette opdateret, og man får besked om at venskabet er oprettet.

Når man klikker på en ven for at skrive sammen, vises samtalen i chatvinduet, dette sker i funktionen `refresh_chat`. Her bliver beskederne mellem brugeren og den valgte ven hentet fra databasen og vist. Hver besked består af 3 dele, hvem der har skrevet beskeden, selve beskeden, og tidspunktet beskeden er sendt på. Programmet viser beskederne i forskellige farve angående hvem der har skrevet beskeden, og afhænger egentlig af hvem af dem der har skrevet. For eksempel så er grøn for egne beskeder og blå er for vennens beskeder.

Fra linje 140 til linje 147 er en lille del af koden, men den er også ret vigtig, da funktionen `auto_refresh` bliver brugt der. Selve det lille stykke kode der tjekker med faste mellemrum, om der er en åbent chat. Hvis der er en chat åben, opdatere den automatisk beskederne ved at kalde `refresh_chat`. Dette gør den ved at sætte sig selv i gang med `self.root.after(...)`, sådan at funktionen bliver kørt igen og igen lidt ligesom en timer.

Nogle ting der også bliver brugt en del er tkinter. Tkinter er et standardbibliotek i Python som bruges til at lave grafiske brugeroverflader (GUI). Denne funktion gør det muligt at lave knapper, vinduer, tekstfelter, lister og meget mere, tkinter har nærmest alt hvad der kunne være behov for når vi snakker om programmer med visuelle overflader. Det er enkelt at bruge og bruges ret godt af mindre programmer såsom chatapplikationer.

`Simpdialog` er en del tkinter, den bruges til at vise små popup-vinduer, hvor brugeren kan indtaste tekst. Den bruges i koden når man gerne vil tilføje en person som ven.

Messagebox er endnu en del af tkinter. Den bruges til at vise beskeder brugeren, disse beskeder kunne fx være fejlmeddelelser, advarsler eller befræftelser. Man kan bruge messagebox til at vise "fejl: brugeren findes ikke" eller beskeden "Du er nu venner med Peter. Dette er med til at give brugeren hurtig og klar feedback om hvad der sker.

Som beskrevet lidt tidligere i dokumentet, når man åbner programmet op, så bliver man fremmødt af en login skærm. Alt den information bliver håndteret af en række funktioner og klasser, som sørger for at gemme og hente brugerdata korrekt.

Test af programmet

Jeg har undervejs hele tiden testet programmet og sørget for at finde fejl og rette dem. Hver eneste lille funktion eller kode der er blevet skrevet ind, har været testet hele tiden. Jeg tester programmet bare jeg ændrede en farve eller noget var skrevet på bare for at sikre at den nye lille ting der blev tilføjet, ikke var skyld i at hele koden gik i stykker. Den måde jeg testede programmet på, var manuelt, grundet at det for mit vedkommende giver mest mening at gøre det.

Jeg lavede manuel testning grundet at det var det der bedst gav mening for mit produkt. En applikation kræver at man manuelt hele tiden sidder og tjekker om tingene virker, og da der ofte kan opstå problematikker så kan det ikke altid være nemt at finde en der vil klikke lidt rundt bare for at finde ud af om programmet virkede som det skulle.

Dog så er der også nogle dele af koden der tester sig selv hvor jeg ikke behøvede at tjekke det manuelt selv, da det egentlig ikke krævede lige så meget og at jeg ville vide om det virkede eller ej med det samme programmet åbnede op. Dette kan man se ved `add_friend` delen i min programmering, hvor den med det samme ville vise fejl hvis man prøvede at add en bruger der ikke eksisterede eller lignende.

En af de mest frustrerende ting ved den her lille real time message app har været når programmet ikke ville starte grundet at der er en lille fejl med et komma eller der mangler et kolon et sted. Der har været så mange ting der har været irriterende og svære at løse. Heldigvis så er det nemt at se hvis man i VSCODE laver en fejl da den altid hvis en hvor fejlen er, selvom fejlen måske kræver mere tid at løse end man lige regnede. Jeg har brugt flere timer på at nogle gange at finde ud af hvordan jeg fik løst fejlen ved at undersøge problematikkerne og se hvad andre har gjort i lignende sekunder.

At der kommer fejl i koden, er svært at undgå når man løser et problem og så tilføjer nye ting til koden fordi man gerne vil have den gøre mere, dette har været med til at det har taget endnu længere tid at færdiggøre et nogenlunde produkt.

Konklusion

Projektet i sig selv har været meget spændende og udfordrende, personligt har det været sjovt at prøve noget lidt anderledes end hvad jeg har været vant til hvad der gælder programmering, da jeg personligt er mere vant til robotprogrammering. Så at vælge noget som at lave en applikation har været meget spændende og interessant for mit eget vedkommende, selvom det måske ikke har været

sjovt alle dage. Det har været et ret hårdt projekt for mig grundet jeg har døjet meget med sygdom og været sygemeldt, hvor jeg derefter kom tilbage til skolen og startede på 4 projekter, så der har været meget jonglering i gang fra min side af, hvor noget af tiden der var planlagt til det her projekt ikke blev til noget. Det har været meget svært for mig at holde en logbog og generelt skrive ind de ting jeg laver når jeg laver dem grundet at jeg arbejder om natten for det mest, hvilket også tydeligt har været det bedste for mig hvad der galt at arbejde med det her projekt.

Det har helt klart været bedst for mig når jeg kunne arbejde for mig selv på de tidspunkter der passer mig bedst, dog så er der nogle ting der er blevet negleret grundet det, såsom logbogen som skrevet før.

Der er mange ting der kunne forbedres når vi snakker om det her projekt at have mere tid kunne have hjulpet en del, være bedre til at planlægge hvornår jeg arbejdede og hvor meget jeg arbejdede ad gangen. Til næste gang tænker jeg det kunne være en god ide at blive bedre til at bruge en logbog hvad der gælder større projekter.

Jeg har lært rigtig meget af det her projekt rent personligt, og omkring de ting jeg burde gøre bedre i fremtiden med lignende projekter. Men en ting jeg har lært, har i den grad været hvor lang tid det kan tage at programmere en applikation og hvor frustrerende det kan være også. Der er så mange ting der kan gå galt, et komma et forkert sted og koden og programmet bliver mærkelig, eller vil slet ikke åbne op igen efter.

Bilag

```
1 import tkinter as tk
2 from tkinter import simpledialog, messagebox
3 import json
4 import os
5 from datetime import datetime
6
7 DATA_FILE = "users_data.json"
8
9 def load_users():
10     if os.path.exists(DATA_FILE):
11         with open(DATA_FILE, "r") as f:
12             return json.load(f)
13     return {}
14
15 def save_users(data):
16     with open(DATA_FILE, "w") as f:
17         json.dump(data, f)
18
19 class ChatApp:
20     def __init__(self, root):
21         self.root = root
22         self.root.title("ChatApp")
23         self.root.geometry("750x550")
24         self.root.configure(bg="#202225")
25         self.root.resizable(False, False)
26         self.current_user = None
27         self.current_chat = None
28         self.refresh_interval = 1000
29         self.login_screen()
30
31     def login_screen(self):
32         self.clear()
33         tk.Label(self.root, text="Velkommen til ChatApp", font=("Helvetica Neue", 20, "bold"), fg="FFFFFF", bg="#202225").pack(pady=30)
34         self.username_entry = tk.Entry(self.root, font=("Helvetica Neue", 14), width=30, bg="#2f3136", fg="FFFFFF", insertbackground="white", relief="flat")
35         self.username_entry.pack(pady=10)
36         tk.Button(self.root, text="Log ind / Opret", command=self.login, font=("Helvetica Neue", 12), bg="#72890A", fg="white", relief="flat").pack(pady=15)
37
38     def login(self):
39         name = self.username_entry.get().strip()
40         if not name:
41             messagebox.showerror("Fejl", "Indtast et brugernavn")
42             return
43         users = load_users()
44         if name not in users:
45             users[name] = {"friends": [], "messages": []}
46             save_users(users)
47         self.current_user = name
48         self.main_screen()
```

Bilag 1

```
49
50 def main_screen(self):
51     self.clear()
52     self.left_frame = tk.Frame(self.root, width=220, bg="#2f3136")
53     self.left_frame.pack(side="left", fill="y")
54     self.right_frame = tk.Frame(self.root, bg="#36393f")
55     self.right_frame.pack(side="right", fill="both", expand=True)
56
57     tk.Label(self.left_frame, text=f"Bruger: {self.current_user}", bg="#2f3136", fg="white", font=("Helvetica Neue", 12)).pack(pady=15)
58     self.friend_listbox = tk.Listbox(self.left_frame, bg="#23272a", fg="white", font=("Helvetica Neue", 11), selectbackground="#72890A", relief="flat", activestyle="none")
59     self.friend_listbox.pack(fill="both", expand=True, padx=10, pady=5)
60     self.friend_listbox.bind("<<ListboxSelect>>", self.select_chat)
61
62     tk.Button(self.left_frame, text="Tilføj ven", command=self.add_friend, font=("Helvetica Neue", 11), bg="#43b581", fg="white", relief="flat").pack(pady=10)
63
64     self.chat_display = tk.Text(self.right_frame, bg="#2c2f33", fg="white", font=("Helvetica Neue", 11), state="disabled", wrap="word", relief="flat", padx=10, pady=10)
65     self.chat_display.pack(fill="both", expand=True, padx=10, pady=(10, 0))
66
67     self.chat_entry = tk.Entry(self.right_frame, font=("Helvetica Neue", 11), bg="#404448", fg="white", insertbackground="white", relief="flat")
68     self.chat_entry.pack(fill="x", padx=10, pady=10)
69     self.chat_entry.bind("<Return>", self.send_message)
70
71     self.update_friend_list()
72     self.auto_refresh()
73
74     def add_friend(self):
75         users = load_users()
76         friend = simpledialog.askstring("Tilføj ven", "Brugernavn på ven:")
77         if not friend:
78             return
79         if friend == self.current_user:
80             messagebox.showwarning("Fejl", "Du kan ikke tilføje dig selv")
81             return
82         if friend not in users:
83             messagebox.showerror("Fejl", "Brugeren findes ikke")
84             return
85         if friend in users[self.current_user]["friends"]:
86             messagebox.showinfo("Info", "I er allerede venner")
87             return
88         users[self.current_user]["friends"].append(friend)
89         users[friend]["friends"].append(self.current_user)
90         users[self.current_user]["messages"].setdefault(friend, [])
91         users[friend]["messages"].setdefault(self.current_user, [])
92         save_users(users)
93         self.update_friend_list()
94         messagebox.showinfo("Tilføjet", f"Du er nu venner med {friend}")
```

Bilag 2


```

95
96 def update_friend_list(self):
97     users = load_users()
98     self.friend_listbox.delete(0, tk.END)
99     for f in users[self.current_user]["friends"]:
100         self.friend_listbox.insert(tk.END, f)
101
102 def select_chat(self, event):
103     sel = self.friend_listbox.curselection()
104     if not sel:
105         return
106     self.current_chat = self.friend_listbox.get(sel)
107     self.refresh_chat()
108
109 def refresh_chat(self):
110     users = load_users()
111     self.chat_display.config(state="normal")
112     self.chat_display.delete(1.0, tk.END)
113     if self.current_chat:
114         for msg_data in users[self.current_user]["messages"].get(self.current_chat, []):
115             try:
116                 sender, content, timestamp = msg_data.split("|")
117             except:
118                 continue
119             tag = "me" if sender == self.current_user else "other"
120             formatted = f"{sender} [{timestamp}]:\n{content}\n\n"
121             self.chat_display.insert(tk.END, formatted, tag)
122             self.chat_display.tag_config("me", foreground="#43B581")
123             self.chat_display.tag_config("other", foreground="#7289DA")
124         self.chat_display.config(state="disabled")
125
126 def send_message(self, event=None):
127     users = load_users()
128     if not self.current_chat:
129         return
130     msg = self.chat_entry.get().strip()
131     if msg:
132         timestamp = datetime.now().strftime("%H:%M")
133         formatted_msg = f"{self.current_user}|{msg}|{timestamp}"
134         users[self.current_user]["messages"][self.current_chat].append(formatted_msg)
135         users[self.current_chat]["messages"][self.current_user].append(formatted_msg)
136         save_users(users)
137         self.chat_entry.delete(0, tk.END)
138         self.refresh_chat()

```

Bilag 3

```

139
140 def auto_refresh(self):
141     if self.current_chat:
142         self.refresh_chat()
143     self.root.after(self.refresh_interval, self.auto_refresh)
144
145 def clear(self):
146     for widget in self.root.winfo_children():
147         widget.destroy()
148
149 if __name__ == "__main__":
150     root = tk.Tk()
151     app = ChatApp(root)
152     root.mainloop()
153

```

Bilag 4